

Laborator 5 - Proiectarea Algoritmilor

METODA BACKTRACKING

1. Problema celor n dame pe tabla de șah.
2. Problema nebunilor pe tabla de șah.
3. Generarea obiectelor combinatoriale:
 - (a) Generarea produsului cartezian al unor mulțimi date.
 - (b) Generarea submulțimilor unei mulțimi date.
 - (c) Generarea aranjamentelor cu repetiție.
 - (d) Generarea aranjamentelor.
 - (e) Generarea permutărilor.
 - (f) Generarea permutărilor cu repetiție.
 - (g) Generarea combinărilor.
 - (h) Generarea combinărilor cu repetiție.
4.
 - (a) Generarea funcțiilor injective definite pe $\{1, 2, \dots, m\}$ cu valori în $\{1, 2, \dots, n\}$.
 - (b) Generarea funcțiilor strict crescătoare definite pe $\{1, 2, \dots, m\}$ cu valori în $\{1, 2, \dots, n\}$.
5. Colorarea unui graf cu n noduri dat cu m culori astfel încât oricare două noduri adiacente să fie colorate diferit.

APLICATII REZOLVATE – METODA BACKTRACKING

UN ȘABLON C++

Următoarea secvență C++ oferă un șablon pentru rezolvarea unei probleme oarecare folosind metoda backtracking. Vom considera în continuare următoarele condiții externe: $x[k] = \overline{A, B}$, pentru $k = \overline{1, n}$. În practică A și B vor avea valori specifice problemei:

```
#include <fstream>

using namespace std;

int x[10] ,n;

int Solutie(int k){
    // x[k] verifică condițiile interne
    // verificare dacă x[] reprezintă o soluție finală
    return 1; // sau 0
}

int OK(int k){
    // verificare conditii interne
    return 1; // sau 0
}

void Afisare(int k)
{
    // afișare/prelucrare soluția finală curentă
}

void Back(int k){
    for(int i = A ; i <= B ; ++i)
    {
        x[k]=i;
        if( OK(k) )
```

```

        if(Solutie(k))
            Afisare(k);
        else
            Back(k+1);
    }
}

int main(){
    //citire date de intrare

    Back(1);

    return 0;
}

```

De multe ori condițiile de existență a soluției sunt simple și nu se justifică scrierea unei funcții pentru verificarea lor, ele putând fi verificate direct în funcția Back().

De cele mai multe ori, rezolvarea unei probleme folosind metoda backtracking constă în următoarele:

1. stabilirea semnificației vectorului soluție;
2. stabilirea condițiilor externe;
3. stabilirea condițiilor interne;
4. stabilirea condițiilor de existență a soluției finale;
5. completarea adecvată a șablonului de mai sus!

APLICATIE: *Să se genereze toate funcțiile surjective $f: \{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, m\}$.*

```
#include <iostream>

using namespace std;

int x[20],m,n;

void afisare()
{
    for(int i=1;i<=n;i++)
    {
        cout <<"f("<<(i)<<")="<<x[i];
        cout <<"\n";
    }
    cout <<"\n";
}

int OK(int k)
{
    int sw=1,i,ap[20]={0};
    for(i=1;i<=k;i++)
        ap[x[i]]=1;
    for(i=1;i<=m;i++)
        if (ap[i]==0) sw=0;
    return sw;
}

void back(int k)
{
    for (int i=1;i<=m;i++)
    {
```

```

x[k]=i;
if(k==n)
{if(OK(k))
afisare();
}
else back(k+1);
}
}
int main()
{
cout <<"n="; cin >>n;
cout <<"m="; cin >>m;
if (m>n) cout <<"Nu exista functii surjective.";
else back(1);
return 0;
}

```